

Monte-Carlo-Simulation einer Fahrradwerkstatt

Author: Alexander Schulz, Datum: 2025-05-01, Matrikelnummer: 55297

1. Modellbeschreibung

Grundannahmen:

- **Saisonaler Einfluss:** Die Anzahl der Reparaturanfragen pro Tag ist stark abhängig vom Monat. Die Hauptsaison liegt zwischen April und September.
- **Wetterabhängigkeit:** Zusätzlich zum Monat wirkt die durchschnittliche Niederschlagsmenge als zufälliger, aber unabhängiger Einflussfaktor auf die Nachfrage.
- **Zufällige Reparaturzeit:** Die Dauer einer Reparatur variiert gleichmäßig zwischen 1 und 3 Stunden.
- **Zufälliger Bedarf an Ersatzteilen:** Die Anzahl und Art der benötigten Ersatzteile sind probabilistisch verteilt und wirken sich auf die Kosten aus.

2. Zielsetzung der Simulation

Die Monte-Carlo-Simulation wurde entwickelt, um die folgenden Fragestellungen zu beantworten:

1. Wie viele Mitarbeiter sollten pro Monat eingestellt werden, um den Gewinn zu maximieren?
2. Wie wirkt sich eine konstante Mitarbeiteranzahl auf die Bearbeitungszeit und die Kundenzufriedenheit aus?
3. Wie groß ist der Einfluss saisonaler Schwankungen und wetterbedingter Unsicherheiten auf die Auslastung?

Zielfunktionen:

- Maximierung des monatlichen Gewinns
- Minimierung der Leerlaufzeit der Mitarbeitenden
- Minimierung der Wartezeit für Kunden (als Proxy für Zufriedenheit)

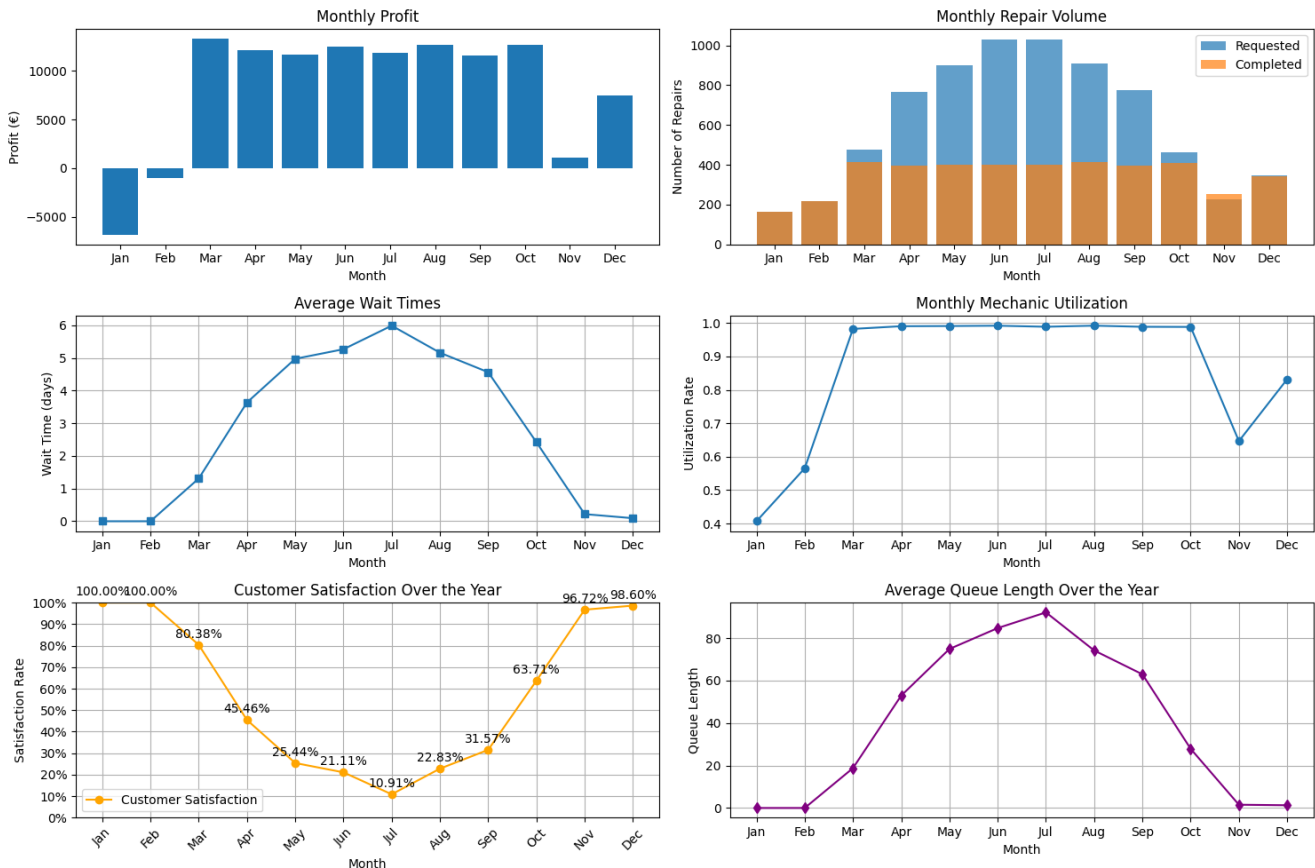
3. Ergebnisse der Simulation

Szenario A: Konstante Mitarbeiteranzahl (5 MA/Jahr)

- In der Hochsaison (Sommermonate) kommt es zu deutlichen Wartezeiten und Umsatzeinbußen

durch nicht bearbeitbare Anfragen.

- In der Nebensaison (Wintermonate) ist die Werkstatt deutlich unterausgelastet.
- **Gesamter Jahresgewinn: €99,090.10**
- **Durchschnittliche Kundenzufriedenheit: 58.06%**
- **Durchschnittliche Wartezeit: 2.8 Tage**

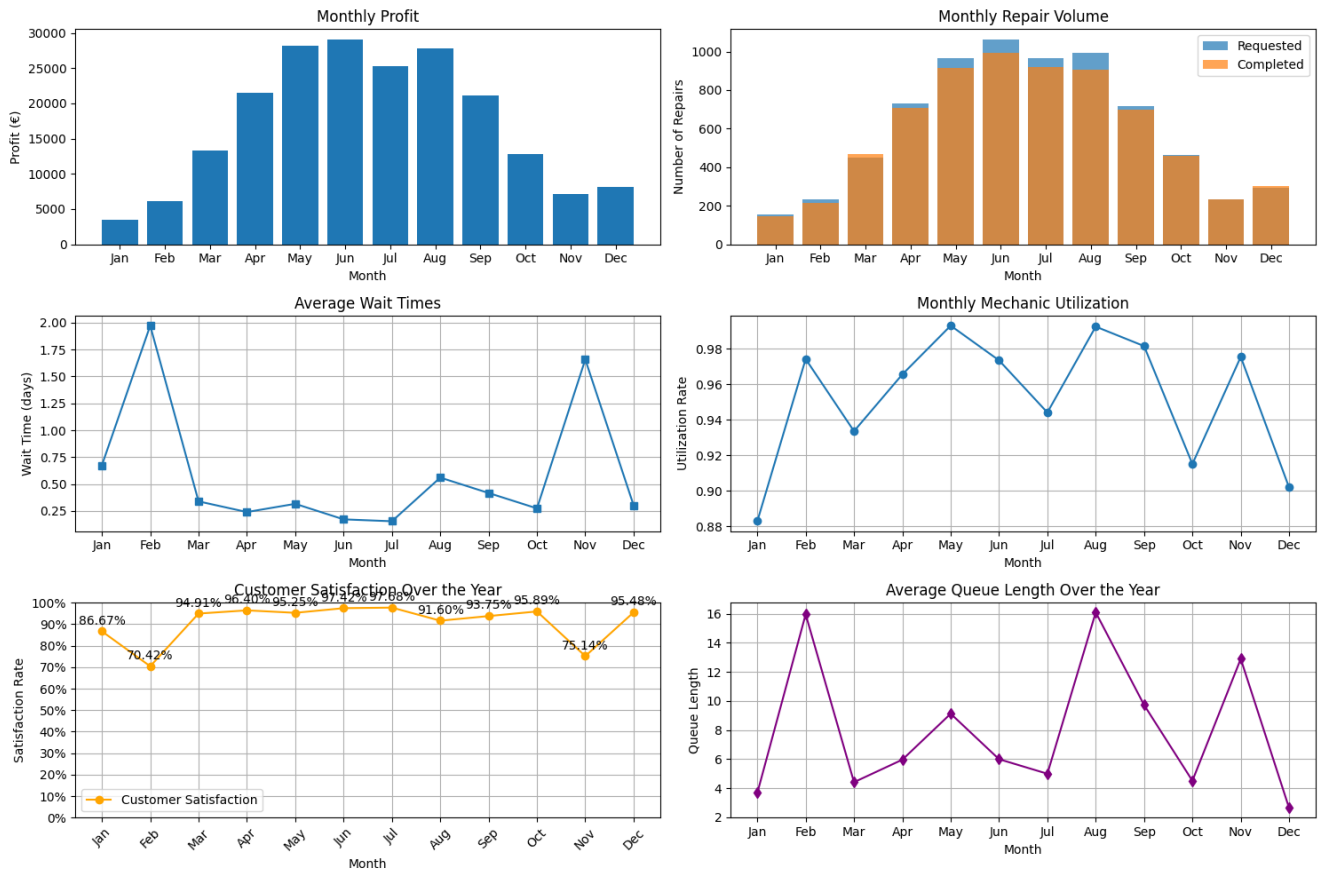


Szenario B: Dynamischer Personaleinsatz nach Bedarf

Optimale monatliche Mitarbeiteranzahl laut Simulation:

Jan	Feb	Mär	Apr	Mai	Jun	Jul	Aug	Sep	Okt	Nov	Dez
2	3	6	9	11	13	12	11	9	6	3	4

- Gleichmäßigere Auslastung über das Jahr
- Geringere Wartezeiten
- Höchster Jahresgewinn im Vergleich
- **Gesamter Jahresgewinn: €204,007.34**
- **Durchschnittliche Kundenzufriedenheit: 90.88%**
- **Durchschnittliche Wartezeit: 0.6 Tage**



4. Schlussfolgerung

- Die Reparatur-Nachfrage ist stark saisonabhängig und zusätzlich wetterbeeinflusst.
- Ein konstanter Personalbestand führt zu Ineffizienzen und verpasstes Umsatzpotenzial.
- Durch monatlich angepasste Mitarbeiterzahlen kann der Gewinn maximiert und die Kundenzufriedenheit deutlich gesteigert werden.
- Die Investition in flexible Personalplanung ist wirtschaftlich sinnvoll.

5. Technische Details

Simulationselemente:

- **Anzahl Reparaturanfragen/Tag:**
 - Poisson-verteilte Anfragen basierend auf saisonalem Monatsmittel
 - Zufällige Wettermodifikation des Tagesbedarfs (Niederschlagsmenge als normalverteilter Störfaktor)
- **Reparaturdauer:**
 - Gleichverteilung zwischen 1 und 3 Stunden
- **Ersatzteilbedarf pro Reparatur:**
 - Zufällig bestimmt (z.B. 0–3 Teile, je mit unterschiedlichen Kosten)

Auszug aus dem Python-Code:

Saisonale Anfrageverteilung:

```
        # Generate random precipitation for the day
        precipitation = np.random.normal(
            self.monthly_avg_precipitation[month]/self.days_in_month[
month],
            self.monthly_avg_precipitation[month]/100 # std 1% avg
precipitation for the day
        )

        # Generate repair requests for the day
```

Generierung von Reparaturanfragen:

```
def generate_daily_repair_requests(self, month, precipitation):
    # Base request rate for this month
    base_requests = self.monthly_base_repairs[month]

    # Weather effect (independent from seasonality)
    avg_daily_precip = self.monthly_avg_precipitation[month] / self.days_in_month
[month]
    precip_delta = precipitation - avg_daily_precip
    weather_factor = 1.0 + (precip_delta * self.precipitation_impact_factor)

    # Calculate expected requests and add randomness
    expected_requests = base_requests * weather_factor
    actual_requests = np.random.poisson(expected_requests)

    return max(0, actual_requests)
```

Reparaturdauer-Verteilung:

```
def generate_repair_time(self):
    # Generate repair time (min 1 hour)
    repair_time = np.random.normal(self.repair_time_mean, self.repair_time_std)
    return max(1.0, repair_time)
```

Ersatzteile-Zufallsgenerator:

```
def determine_parts_requirement(self):
    # Randomly determine parts requirement category
    rand = np.random.random()
    if rand < self.parts_probabilities["basic"]:
```

```
        return "basic"
    elif rand < self.parts_probabilities["basic"] + self.parts_probabilities[
"medium"]:
        return "medium"
    else:
        return "major"
```

Anhang

Der vollständige Python-Code und die Ergebnisse können als ZIP-Datei zur Verfügung gestellt werden. Bitte ggf. per Mail anfordern.